

Robotic Indoor Path Planning using Dijkstra's Algorithm with Multi-Layer Dictionaries

Syed Abdullah Fadzli, Sani Iyal Abdulkadir, Mokhairi Makhtar, Azrul Amri Jamal

Faculty of Informatics & Computing
Universiti Sultan Zainal Abidin
Terengganu, Malaysia

Abstract— Dijkstra's algorithm is a classic algorithm for finding the shortest path between two points due to its optimisation capability. The adjacency matrix is the naïve storage structure of the algorithm. This storage structure has limited the use of the algorithm as it expands large storage space. A multi-layer dictionary is proposed in this work to enhance the storage structure. Previously, the algorithm was used to optimise single parameter (such as distance, time and fuel) for movement between two places. The path computed using the classic Dijkstra's algorithm is the shortest; however, it may not be the most feasible. The algorithm needs to optimise other factors such as energy consumption and the degree of turns a robot need to take. Junction degree of difficulty function is introduced to further improve the output. It is defined as the degree of difficulty of the path between two points in an open environment. The experimental result shows that the proposed path planning method produced the most optimal path between two points when applied to a map of any indoor terrain.

Keywords— Dijkstra's algorithm, indoor path planning, adjacency matrix, multi-layer dictionary

I. INTRODUCTION

Path planning is an act of finding the shortest, most feasible and most optimal path for an autonomous vehicle that enables it to move from one point to another. Given a map with a source and destination points, path planning includes locating a trajectory that will result in the robot to reach the target point when executed. The result of this algorithm is then utilised for physical movement for the autonomous robot on the sought path [1]. The environment may either be indoors or outdoors. The indoor environment is characterised by the presence of surrounding walls, doorways, corridors, furniture, people and other moveable objects. The static indoor environment is always considered to contain only fixed obstacles within the terrain. The consideration was due to the assumption that the robot moves along a dedicated part of the territory that will never be obstructed by other moving objects. Numerous works were accomplished to address this issue [2] [3] [4] [5] [6]. Dijkstra's algorithm is among the most famous solutions regarding path planning. However the naïve storage structure of the algorithm provides limitations to an indoor environment which needs more information for each nodes other than just location. Many researchers proposed various methods to improve the storage structure of the algorithm. [7] had proposed an improvement to the Dijkstra's data storage structure and the node search method to improve the efficiency of the algorithm in a geographic information system. An

object-oriented approach was applied to the data storage structure to address the issue of large memory utilisation caused by an adjacency matrix. Based on the improved storage structure, the heap structure was adopted to store nodes, which can minimise storage space and reduce node finding time.

However, the building of the heap and its linear search requires complex work and time; it makes the algorithm difficult to be employed. [8] had considered improving traditional Dijkstra's algorithm in terms of storage structure and search area in road networks route planning. Adjacency-list was applied to the storage structure to reduce unnecessary utilisation of memory space caused by the adjacency matrix storage structure implementation. They further reduced the searching area of the algorithm by creating a rectangle with a diagonal line from that source to the destination points within the searching area of the classical Dijkstra's algorithm; this increased the real-time performance of route planning. However, although the algorithm managed to increase the performance, it reduced efficiency by restricting the search area. As a result, there might be other path which are better than the proposed path, but they were ignored by the search algorithm.

This paper proposed a multi-layer dictionary storage structure for Dijkstra's algorithm. It also considered the difficulty of passing junctions between two points as an additional parameter for computing the optimal path of the mobile robot. Experiment is done by comparing the optimum path (later will be referred to as OP) produced by the proposed method and the shortest path (later will be referred to as SP) produced by the traditional Dijkstra's algorithm.

II. DIJKSTRA'S ALGORITHM

Dijkstra is a breadth-first-search (BFS) algorithm for finding the shortest paths from a single source vertex to all other vertices. It processes vertices in increasing order of their distance from the source, which are also called root vertices. The shortest path between two vertices is a path with the shortest length (i.e. least number of edges), also called link-distance.

Let $G = (U, V)$ be a weighted undirected graph, with weight function $w: E \rightarrow R$ mapping edges to real-valued weight. If $e(u, v)$, then we write $w(u, v)$ for $w(e)$.

- The length of a path $p = \langle v_0, v_1, v_2, v_3 \dots v_k \rangle$ would be the total of the weight of its constituent edges as in (1):

$$\text{length}(p) = \sum_{i=1}^k w(v_{i-1}, v_i) \quad (1)$$

- The distance from u to v , denoted by $\delta(u, v)$, is the length of the minimum path if there is a path from u to v ; and ∞ is otherwise.

The general idea of Dijkstra's algorithm is to report vertices in increasing order of their distances from the source vertex while constructing the shortest path tree edge by edge; at each step adding one new edge, corresponding to the construction of the shortest path to the current new vertex. This is accomplished in the following steps:

- Maintain an estimate $d[v]$ of the $\delta(s, v)$ of the shortest path for each vertex v .
- Always $d[v] \geq \delta(s, v)$ and $d[v]$ equals the length of a known path ($d[v] = \infty$ if we have no path so far).
- Initially, $d[s] = 0$ and all other $d[v]$ values are set ∞ . The algorithm will then process the vertices one by one in some order. The processed vertex's estimate will be validated as being the real shortest distance; i.e. $d[v] = \delta(s, v)$.

The term "processing a vertex u " means finding new paths and updating $d[v]$ for all $v \in \text{adj}[u]$ if necessary. The process by which an estimate is updated is called relaxation. When all vertices have been processed, $d[v] = \delta(s, v)$ for all v .

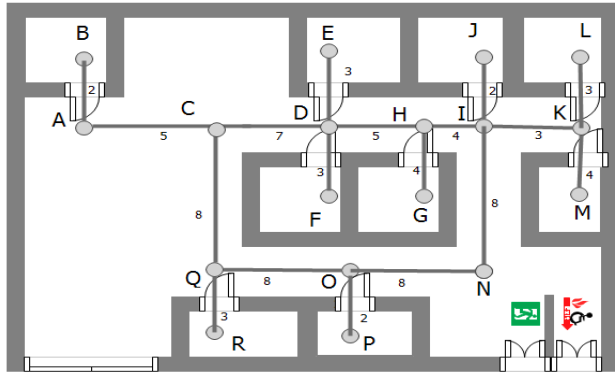


Fig. 1. Floor plan of an indoor terrain.

Given Fig. 1 as a floor plan of an indoor environment, a multi-layer dictionary can be applied to store the data. The floor plan consists of rooms and hallways connected by links that serve as a navigation path of a mobile robot within the terrain. These links consist of points of interest and junctions that the robot will stop at or passes through when moving from a starting point to an end point. The links form a network of all paths within the environment. Dijkstra's algorithm can be applied to find the shortest path between any two locations within the terrain.

III. MULTI-LAYER DICTIONARY

The multi-layer dictionary consists of two dictionaries and a list of data structure organised in hierarchical order. The first dictionary maps each and every node (i.e. location) to its neighbouring nodes. The second dictionary stores the path information of each neighbouring pathway. Given a map or floor plan of an indoor environment as G with two neighbouring nodes labelled as U and V , the expression $G[U]$ will return all the neighbours of the node U , and $G[U][V]$ will return the path information between node U and V information.

Let us apply the multi-layer dictionary to store the data of the floor plan in Fig. 1 and use Dijkstra's algorithm to compute the shortest path between two predetermined locations. The plan consists of locations of interest labelled from A to R and the distance between every two nodes are directly linked from one another. This information can be transformed into a graph. The location can serve as nodes, and distances can act as edges of the graph. Since the locations of each node are known, their movement orientation can be pre-determined and stored in the dictionary.

Source	Destination	Path Info		
A	B	Distance	Starting Orientation	Ending Orientation
	C	Distance	Starting Orientation	Ending Orientation
B	A	Distance	Starting Orientation	Ending Orientation
	C	Distance	Starting Orientation	Ending Orientation
C	A	Distance	Starting Orientation	Ending Orientation
	D	Distance	Starting Orientation	Ending Orientation
D	Q	Distance	Starting Orientation	Ending Orientation
	C	Distance	Starting Orientation	Ending Orientation
	E	Distance	Starting Orientation	Ending Orientation
	F	Distance	Starting Orientation	Ending Orientation
	H	Distance	Starting Orientation	Ending Orientation

Fig. 2. Multi-layer dictionary representing the floor plan.

Fig. 2 displayed the basic information obtained from the plan. It consists of three columns, and several rows that are equal to the number of nodes on the plan. The first column consists of all nodes exist on the graph. This serves as the key to the first dictionary. The second column consists of all linked nodes on the graph. This serves as the values of the second dictionary. The third column is a sorted list that consists of three (3) information; i.e. distance between the two nodes, starting orientation and end orientation of two neighbouring nodes. The first and second columns make the first dictionary, the second and third columns make the second dictionary, while the value of the second dictionary is itself a list. For example, from node A to C, the distance is 5 meters, starting orientation is 90 degrees and ending orientation is also 90 degrees. While from node C to Q, the distance is 6 meters, starting orientation is 180 degrees and ending orientation is also 180 degrees. These information can be used to determine the starting and ending orientation of the mobile robot at each

node. For instance, when a mobile robot wants to travel from A to Q through C, it needs to make a turn at node C. By calculating the different between ending orientation 90 degrees for path A-C and starting orientation of 180 degrees for path C-Q, the robot will make a RIGHT turn of 90 degrees at junction C.

IV. IMPLEMENTATION OF MULTI-LAYER DICTIONARY IN FINDING ROBOTIC PATH

Dijkstra's algorithm was adopted to determine the best path between two points using the multi-layer dictionary floor plan data. The objective of the proposed method is to find the shortest and the easiest path for the robot which is called the optimum path. Easiness is defined by less total degree angle of turns the robot need to take along the path, from starting point to end point. A turn procedure, whether to go right of or to go left at each node is considered a problem for a robot because it needs to slow down the movement and carefully execute the turn procedure.

TABLE I. OUTPUT OF THE PROPOSED ALGORITHM

Start and end points	Results of Dijkstra's algorithm	Result of the algorithm using dictionary
L to R	<i>Shortest Path:</i> L, K, I, N, O, Q, R <i>Orientations:</i> STRAIGHT, RIGHT, LEFT, RIGHT, STRAIGHT, STRAIGHT, LEFT <i>Total length:</i> 33 meters <i>Total degree of turns:</i> 360 degrees	<i>Optimal Path:</i> L, K, I, H, D, C, Q, R <i>Orientations:</i> STRAIGHT, RIGHT, STRAIGHT, STRAIGHT, STRAIGHT, STRAIGHT, LEFT, STRAIGHT <i>Total length:</i> 33 meters <i>Total degree of turns:</i> 180 degrees
A to N	<i>Shortest Path:</i> A, C, Q, O, N <i>Orientations:</i> STRAIGHT, RIGHT, LEFT, STRAIGHT <i>Total length:</i> 29 meters <i>Total degree of turns:</i> 180 degrees	<i>Optimal Path:</i> A, C, D, H, I, N <i>Orientations:</i> STRAIGHT, STRAIGHT, STRAIGHT, STRAIGHT, RIGHT <i>Total length:</i> 29 meters <i>Total degree of turns:</i> 90 degrees
E to P	<i>Shortest Path:</i> E, D, C, Q, O, P <i>Orientations:</i> STRAIGHT, RIGHT, LEFT, LEFT, RIGHT <i>Total length:</i> 28 meters <i>Total degree of turns:</i> 180 degrees	<i>Optimal Path:</i> E, D, C, Q, O, P <i>Orientations:</i> STRAIGHT, RIGHT, RIGHT, LEFT, LEFT, RIGHT <i>Total length:</i> 28 meters <i>Total degree of turns:</i> 180 degrees

The result of the algorithm was then applied to the motor controller to move the robot along the path. Note that the result of the algorithm were commands that direct the robot to choose between LEFT, RIGHT or STRAIGHT procedures at each

node. As the result, the algorithm will guided the robot to reach the destination from the starting point to end point through the proposed path. A path is a connection of series of edge computed as (2):

$$path(x, y) = \min \sum (w(d) + w(DoD)) \quad (2)$$

where path (x, y) was the most optimal path from node x to y ; $w(d)$ is the distance weight; and $w(DoD)$ is the weight function of the degree of difficulty of passing an edge of two neighbouring nodes.

Table 1 compares the different between the shortest path (SP) produced by the traditional Dijkstra's algorithm and the optimal path (OP) produced by the proposed Dijkstra's using multi-layer dictionary. In all the results, we calculated the total degree of turns which the robot need to take at each nodes along the path. Both RIGHT and LEFT procedures are considered as a 90 degrees angle of turn, while STRAIGHT is considered as 0 degree angle of turn. A path with a small total degree angle of turn is considered as an easier path compared a bigger total degree angle of turn because they need to execute less turn procedures.

The results shows that in all 3 cases, the SP and the OP produced by the traditional Dijkstra's algorithm and the proposed method have the same lengths. However, it is observed that the OP produced have less total degree angle of turns compared to SP which results in a more comfortable and faster pathway for both 'L to R' and 'A to N'. Both OP have a significant different of 180 degrees in case 'L to R' and 90 degrees of turn difference in case 'A to N'. While in the case of 'E to P', both SP and OP are the same pathway with the same length and the same total degree angle of turns.

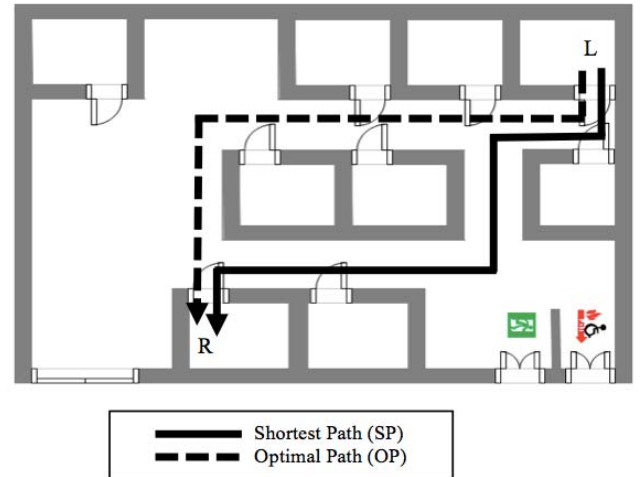


Fig. 3. Shortest path vs. optimum path.

Fig. 3 shows the comparison between SP and OP in the case of 'L to R'. Comparing both path, although the SP passes through only 7 nodes compared to 8 nodes by the OP (refer to Fig. 1), the SP is considered less easy with slower execution

time because of the bigger total degree angle of turns the robot had to execute along the way. The same observation goes to the 'A to N' case where the traditional Dijkstra's algorithm produced the SP which only went through only 5 nodes compared to 6 nodes in the OP produced by the proposed method. However, the OP is considered as an easier path with only 90 degrees angle of turn in total compared to 180 degrees by the SP.

V. CONCLUSION

This paper proposes a robotic path planning using a multi-layer dictionary which provides more comprehensive data structure for Dijkstra's algorithm in an indoor environment application where GPS coordinates and compass orientation are not reliable. The path information in the data structure helps to determine the degree of rotational angle the robot need to execute at each node or junction. Different with the traditional algorithm, the proposed method considered both the length and the degree of difficulty of the path. As the result, the proposed algorithm not only managed to produce the shortest path in term of length, but in addition, the easiest path in terms of the least total degree angle of turns, which is missing in the traditional Dijkstra's algorithm.

The experimental result shows that, compared to the traditional Dijkstra's algorithm, the proposed method produced different paths in 2 cases and the same path in another case, with the same length in all 3 cases. However, it is observed that the optimal path produced by the proposed method have less total degree angle of turns compared to the shortest path produced by the traditional Dijkstra's algorithm, which results in a more comfortable and faster pathway

ACKNOWLEDGEMENT

This research was partially supported by Universiti Sultan Zainal Abidin and Ministry of Higher Education Malaysia (Grant no. RAGS/1/2014/ICT01/UNISZA/1) . We thank all our colleges at Faculty of Informatics for comments that greatly improved the article.

REFERENCES

- [1] Rahul Kala, Anupam Shukla, and Ritu Tiwari, "Robotic Path Planning using Multi Neuron Heuristic Search", in *proceeding of 2nd International Conference on Interaction Sciences: Information Technology, Culture and Human (ICIS '09)*, New York, 2009, pp. 1318-1317
- [2] Hua Wu, A. Marshall, and W. Yu, "Path Planning and Following Algorithms in an Indoor Navigation Model for Visually Impaired," in *proceeding of 2nd International Conference on Internet Monitoring and Protection*, San Jose, CA, 2007, p. 38.
- [3] Kai Sun, Yuanlong Yu, and J. Gu, "Efficient robot navigation for semi-structured indoor storehouse," in *proceeding of 28th Canadian Conference on Electrical and Computer Engineering (CCECE)*, Halifax, NS, 2015, pp. 1313-1317.
- [4] Yukuan Sun, Jianming Wang, and Xiaojie Duan, "Research on path planning algorithm of indoor mobile robot," in *proceeding of International Conference on Mechatronic Sciences, Electric Engineering and Computer (MEC)*, Shengyang, 2013, pp. 1108 - 1111.
- [5] Han Hongling and Yang Fenglei, "Path Planning of an Indoor Mobile Robot Navigated by Infrared," in *proceeding of International Conference on E-Product E-Service and E-Entertainment (ICEEE)*, Henan, 2010, pp. 1-4.
- [6] Hongtai Cheng, Heping Chen, and Yong Liu, "Topological Indoor Localization and Navigation for Autonomous Mobile Robot," *IEEE Transactions on Automation Science and Engineering*, vol. 12, no. 2, pp. 729 - 738, April 2015.
- [7] Y.Q. Liu, Q.L. Huang, Y.X. Zhang, G.F. Luan M.H. Xu, "An improved Dijkstra's shortest path algorithm for sparse network," *Applied Mathematics and Computation*, vol. 185, no. 1, pp. 247-254, February 2007.
- [8] DongKai Fan and Ping Shi, "Improvement of Dijkstra's algorithm and its application in route planning," in *proceeding of 7th International Conference on Fuzzy Systems and Knowledge Discovery (FSKD)*, Yantai, Shandong, 2010, pp. 1901 - 1904.